Test-Time Graph Rebirth For GNN Generalization Under Distribution Shifts

Xin Zheng[1], Bo Li[1], Yu Zheng[1], Qin Zhang[2], Haishuai Wang[3], Yuxuan Liang[4], Alan Wee-Chung Liew[1], Shirui Pan[1]*
[1]Griffith University, Australia, [2]Shenzhen University, China, [3]Zhejiang University, China
[4]The Hong Kong University of Science and Technology (Guangzhou), China
xin.zheng@griffith.edu.au, s.pan@griffith.edu.au

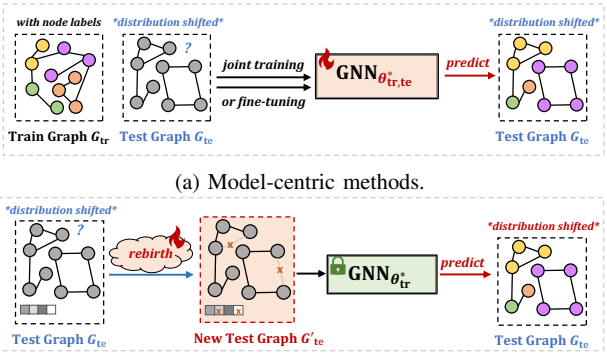
***Abstract*—Recent advancements in test-time adaptation (TTA) offer promising solutions for mitigating performance degradation caused by distribution shifts. However, they may fall short in practical deployment of GNNs during test time, due to the significant reliance on impractical retraining or modifications to online GNN models. To address such challenges, in this work, we propose a novel method, *i.e.*, Test-Time Graph REBirth, dubbed TT-GREB, to effectively generalize the well-trained GNN models to the test-time graphs under distribution shifts by directly manipulating the test graph data. Concretely, we develop an overall framework designed by two principles for obtaining newly reborn test graphs, corresponding to two sub-modules: (1) *prototype extractor* for re-extracting the environment-invariant features of the test-time graph; and (2) *environment refiner* for re-fining the environment-varying features to explore the potential shifts. Furthermore, we propose a dual test-time graph contrastive learning objective with an effective iterative optimization strategy to obtain optimal prototype components and environmental components of the test graph. Extensive experiments on real-world graphs under diverse test-time distribution shifts verify the effectiveness of our proposed method, showcasing its superior ability to manipulate test-time graphs for better GNN generalization ability.**

***Index Terms*—Test-time adaptation, graph distribution shifts, data-centric graph machine learning**

I. Introduction

Recent advances in graph neural networks (GNNs) have achieved great success with promising learning abilities for various graph structural data in numerous real-world applications [1], [2], [3], [4], [5], [6], [7], [8], [9], [10], [11]. Well-designed GNN models are ultimately intended for practical deployment and serving on various graph learning tasks [12], [13], [14], [15], [16], [17], [18]. However, these expertly trained GNNs generally experience significant performance degradation due to the *graph distribution shift* issue between the training and the test graphs [19], [20], [21], [22], [23], [24]. The main reason behind such distribution mismatch lies in the underlying environmental variations, with time-related attribute changes, agnostic corruptions, and inconsistent graph data collection procedures [25], [20], [26]. These factors would lead to considerable differences in node contexts, graph structures, and the overall scale of graphs during test time.

To overcome the model generalization challenge caused by graph distribution shifts, there is a growing focus on research into learning with distribution shifts on graphs, from the *model-centric* perspective [19], [27], [28], [29], as shown in Fig. 1 (a). Typically, these existing methods incorporate invariant representation learning into GNN development through customizing model architectures and training strategies [30], [31], [32], [33]. However, in real-world GNN deployment and serving, redesigning GNN architectures or retraining well-trained models is typically impractical, especially when these GNNs are continuously in service online, making it increasingly difficult to access and modify their parameters.



(a) Model-centric methods.

(b) Test-time data-centric methods (Ours).

Fig. 1: Comparison between model-centric v.s. test-time data-centric graph learning under distribution shifts.

Given all these circumstances, including *complex training-test graph distribution shifts* and *inaccessible online-serving GNN model fine-tuning*, an intriguing problem emerges:

> **Question:** Is it possible to give the rebirth of the test-time graph data from the data-centric perspective, to serve various well-trained GNN models for better generalization under distribution shifts?

In this work, we provide a feasible graph data-centric solution to answer this question and propose a novel **T**est-**T**ime **G**raph **REB**irth method, dubbed **TT-GREB**, by directly manipulating the test-time graph data for enhanced quality, without accessing the training graph data or modifying the well-trained GNNs, as shown in Fig. 1 (b). Specifically, the original test graph would undergo a *rebirth process* to emerge as a new test graph, which is then fed into the well-trained GNN model for direct inference. In the test-time graph rebirth process, we identify

*Corresponding author.

2025 IEEE International Conference on Data Mining (ICDM) | 979-8-3315-9599-9/25/$31.00 ©2025 IEEE | DOI: 10.1109/ICDM65498.2025.00105

2374-8486/25/$31.00 ©2025 IEEE
DOI 10.1109/ICDM65498.2025.00105

Authorized licensed use limited to: Hong Kong University of Science and Technology. Downloaded on September 07,2026 at 07:57:03 UTC from IEEE Xplore. Restrictions apply.

two essential components that decomposed a graph under distribution shifts: (a) *the environment-invariant component* retains consistent informative features (such as node class label prototypes) across various training and test graph distributions; (b) *the environment-varying component* dictates the extent of shifts in graph data distribution between training and test.

Based on these two essential components, our TT-GREB framework is founded on two key principles: (1) re-extracting the environment-invariant features of the test-time graph for identifying the predictive pattern of node class label prototypes with a **prototype extractor**, and (2) re-fining the environment-varying features to explore the potential shifts in the training-test distribution with a **environment refiner**, leading to the alignment of the training-test environment latent space with improved generalization capabilities for online GNN deployment. Furthermore, due to the unknown test-time graph node labels, we propose a dual test-time graph contrastive learning objective with self-supervision signals, along with an effective iterative optimization strategy to obtain expressive prototype features and environmental features of the test graph. In this way, we preserve prototype features to determine the node class label's predictive patterns; meanwhile, we adjust the environmental features of the original test graph to match those of the training graph. Then, these components are reassembled into a newly reborn test graph, which better suits the well-trained GNNs' generalization under distribution shifts. In summary, the contributions of this work are as follows:

- **Graph Data-centric Paradigm.** We introduce a novel graph data-centric paradigm, Test-Time Graph REBirth (TT-GREB), designed to enhance the generalization ability of well-trained GNNs to real-world test graphs experiencing distribution shifts at test time.
- **Innovative Solution.** We develop a comprehensive framework with two core components: a prototype extractor that identifies invariant features within test-time graphs, and an environment refiner that adjusts varying features to align the latent spaces of training and test environments. These components are effectively optimized through a dual test-time graph contrastive learning objective and an iterative optimization strategy.
- **Comprehensive Evaluation.** We evaluate the proposed method on real-world test graphs under diverse graph distribution shifts. Extensive experimental results reveal consistently strong generalization ability on various well-trained GNN models, providing compelling evidence for the efficacy of the proposed method.

II. Related Work

Test-time Adaptation. Our work is related to test-time adaptation methods, which aim to enable dynamic tuning of the pre-trained model to generalize and adapt well to the test samples [34], [35], [36], [37], [38]. The pioneering work is Test-Time Training (TTT) [34], which re-trains model weights at test time on a single test image sample. Moreover, TENT [35] proposes a fully test-time adaptation problem that only accesses model parameters and the test data. GTRANS [26] first proposes to adapt test graph data without accessing the training procedure and GNN architectures. However, GTRANS employs a fully parameterized matrix to represent modified test-time graph node features and engages in graph structure learning within the dynamic node representation learning process. It uses a fully parameterized approach and a binary-space projected gradient descent method for test-time graph learning, narrowing the scope of test graph learning and limiting the flexibility in handling diverse graph structures. Besides, a very recent test-time adaptation method, MATCHA [39], mainly focuses on graph structural shift specifically for the graph homophily property. In this work, we conduct the parameterized decomposition of the test graph to give it a rebirth by re-extracting the invariant features and re-fining test-time varying features of test graphs under diverse graph distribution shifts.

Graph Learning Under Distribution Shifts. Our work is also relevant to the research topic of graph learning under distribution shifts, whose goal is to develop a GNN model for better generalization ability on test graphs under graph distribution shifts [29], [20], [25], [40], [41], [42]. Typically, UDAGCN [29] conducts unsupervised graph domain adaptation with a domain adversarial method to learn domain-invariant embeddings across the source domain and the target domain. AIA [25] proposes graph data augmentation with effective GNN learning to handle the covariate shift on graphs for the graph classification task. Different from these graph model-centric methods, in this work, we mainly focus on modifying the test graph data with self-supervision signals to deal with the distribution-shifted test graph learning problem.

III. Test-time Graph Rebirth (TT-GREB)

Preliminary. Given a training graph $G_{\text{tr}} = (\mathbf{X}_{\text{tr}}, \mathbf{A}_{\text{tr}}, \mathbf{Y}_{\text{tr}}) \sim P_{\text{tr}}$ with N nodes and C-classes of node labels, where $\mathbf{X}_{\text{tr}} \in \mathbb{R}^{N\times d}$ is the d-dimension nodes feature matrix indicating node attribute semantics, $\mathbf{A}_{\text{tr}} \in \mathbb{R}^{N\times N}$ is the adjacency matrix indicating whether nodes are connected or not by edges with $\mathbf{A}_{\text{tr}}^{i,j} = \{0,1\} \in \mathbb{R}$ for i-th and j-th nodes, $\mathbf{Y}_{\text{tr}} \in \mathbb{R}^{N\times C}$ denotes the node labels, and P_{tr} is the training graph distribution.

Training Stage: A GNN model is trained on G_{tr} according to the following objective function for node classification:

$$\begin{aligned} &\boldsymbol{\theta}^*_{\text{tr}} = \min_{\boldsymbol{\theta}_{\text{tr}}} \mathcal{L}_{\text{cls}}\left(\hat{\mathbf{Y}}_{\text{tr}}, \mathbf{Y}_{\text{tr}}\right), \text{ where} \\ &\mathbf{Z}_{\text{tr}}, \hat{\mathbf{Y}}_{\text{tr}} = \text{GNN}_{\boldsymbol{\theta}_{\text{tr}}}(\mathbf{X}_{\text{tr}}, \mathbf{A}_{\text{tr}}). \end{aligned} \tag{1}$$

The parameters of GNN trained on G_{tr} is denoted by $\boldsymbol{\theta}_{\text{tr}}$, $\mathbf{Z}_{\text{tr}} \in \mathbb{R}^{N\times d_1}$ is the output node embedding of graph G_{tr} from $\text{GNN}_{\boldsymbol{\theta}_{\text{tr}}}$, and $\hat{\mathbf{Y}}_{\text{tr}} \in \mathbb{R}^{N\times C}$ denotes the output node labels predicted by the trained $\text{GNN}_{\boldsymbol{\theta}_{\text{tr}}}$. By optimizing the node classification loss function $\mathcal{L}_{\text{cls}}$ (*e.g.*, cross-entropy loss) between GNN predictions $\hat{\mathbf{Y}}_{\text{tr}}$ and ground-truth node labels $\mathbf{Y}_{\text{tr}}$, the GNN model that is well-trained on G_{tr} can be denotes as $\text{GNN}_{\boldsymbol{\theta}^*_{\text{tr}}}$ with optimal weight parameters $\boldsymbol{\theta}^*_{\text{tr}}$. Note that once we obtain the optimal $\text{GNN}_{\boldsymbol{\theta}^*_{\text{tr}}}$ that has been well-trained on G_{tr}, the GNN model would be **fixed** and G_{tr} would not be accessible during test time.

Authorized licensed use limited to: Hong Kong University of Science and Technology. Downloaded on September 07,2026 at 07:57:03 UTC from IEEE Xplore. Restrictions apply.

Test-time Inference: For practical GNN deployment and serving, given a real-world test graph $G_{\text{te}} = (\mathbf{X}_{\text{te}}, \mathbf{A}_{\text{te}}) \sim P_{\text{te}}$ including M nodes with its feature matrix $\mathbf{X}_{\text{te}} \in \mathbb{R}^{M\times d}$ and its adjacency matrix $\mathbf{A}_{\text{te}} \in \mathbb{R}^{M\times M}$, we assume that there are potential distribution shifts between G_{tr} and G_{te}, which mainly lies in node contexts, graph structures, and scales as $P_{\text{tr}} \neq P_{\text{te}}$, but the label space keeps consistent under the covariate shift, *i.e.*, all nodes in G_{te} are constrained in the same C-classes as G_{tr} as $\{\mathbf{Y}_{\text{tr}}, \mathbf{Y}_{\text{te}}\} \in \mathcal{Y} = \{1, \cdots, C\}$. Generally, the well-trained model $\text{GNN}_{\boldsymbol{\theta}^*_{\text{tr}}}$ would be directly used for inferring on the test graph as $\hat{\mathbf{Y}}_{\text{te}} = \text{GNN}_{\boldsymbol{\theta}^*_{\text{tr}}}(\mathbf{X}_{\text{te}}, \mathbf{A}_{\text{te}})$. However, due to the latent graph distribution shifts at the test time, the optimal parameters $\boldsymbol{\theta}^*_{\text{tr}}$ learned on the training graph would not be ideal for inference on the test-time graph. Hence, instead of learning optimal GNN parameter $\boldsymbol{\theta}^*_{\{\text{tr,te}\}}$ with GNN architecture re-designing and fine-tuning, in this work, we pay attention to a data-centric solution through modifying the test graph G_{te} under distribution shifts by test-time graph rebirth, without re-designing and fine-tuning the well-trained $\text{GNN}_{\boldsymbol{\theta}^*_{\text{tr}}}$, and without accessing the training graph G_{tr}.

A. Problem Formulations

Through the length of graph structural data generation hypothesis in existing studies [43], [44], [45], [25], [46], [20], a graph can be generated through a mapping $f_{\text{gen}} : \{\mathcal{C}, \mathcal{S}\} \to \mathcal{G}$, where $\mathcal{C} \subseteq \mathbb{R}^{n_c}$ and $\mathcal{S} \subseteq \mathbb{R}^{n_s}$ are the latent variables denotes the environment-invariant part and the environment-varying parts for generating the graph $G \in \mathcal{G} = \cup_{N=1}^{\infty}\{0,1\}^N \times \mathbb{R}^{N\times d}$, where n_c and n_s denote the dimensions of latent variables, respectively. Inspired by such structural causal model (SCM) [46], [20] for graph generation progress, we have the following proposition to comprehend test-time graph distribution shifts and the test-time graph discrepancy from the training graph.

Proposition III.1. *Given the training graph $G_{tr} \sim P_{tr}$, there exist the environment-invariant part G_{tr}^{Inv} and the environment-varying part G_{tr}^{Env} components in this graph, denoting as $G_{tr} = \{G_{tr}^{Inv}, G_{tr}^{Env}\}$, likewise for the the test graph $G_{te} \sim P_{te}$ with $G_{te} = \{G_{te}^{Inv}, G_{te}^{Env}\}$. Then, the GNN model $GNN_{\boldsymbol{\theta}^*_{tr}}$ that has been well trained on the G_{tr} would keep good generalization on the test graph when*

$$\begin{aligned} &G_{tr}^{Inv} = G_{te}^{Inv} \sim Q^{Inv}, \quad dist\left(G_{tr}^{Env}, G_{te}^{Env}\right) < \epsilon, \\ &\text{where } G_{tr}^{Env} \neq G_{te}^{Env}, G_{tr}^{Env} \sim Q_{tr}^{Env}, G_{te}^{Env} \sim Q_{te}^{Env}. \end{aligned} \tag{2}$$

In this proposition, the function $dist(\cdot)$ quantifies the discrepancy between the components of the training graph and the test graph that vary with the environment. Additionally, Q^{Inv} represents the distribution of latent variables that remain constant across different environments, and is expected to be identical in both the training and test graphs. Furthermore, the environmental variables of $G_{\text{tr}}^{\text{Env}}$ and $G_{\text{te}}^{\text{Env}}$ are presumed to adhere to distinct, environment-specific distributions, denoted as $Q_{\text{tr}}^{\text{Env}}$ and $Q_{\text{te}}^{\text{Env}}$, respectively.

Proof. Given the training objective of $\text{GNN}_{\boldsymbol{\theta}_{\text{tr}}}$ in Eq. (1) with $\mathcal{L}_{\text{cls}}(G_{\text{tr}}; \boldsymbol{\theta}_{\text{tr}})$, it can be decomposed into:

$$\mathcal{L}_{\text{cls}}(G_{\text{tr}}; \boldsymbol{\theta}_{\text{tr}}) = \mathcal{L}^{\text{Inv}}(G_{\text{tr}}^{\text{Inv}}; \boldsymbol{\theta}_{\text{tr}}) + \mathcal{L}^{\text{Env}}(G_{\text{tr}}^{\text{Env}}; \boldsymbol{\theta}_{\text{tr}}), \tag{3}$$

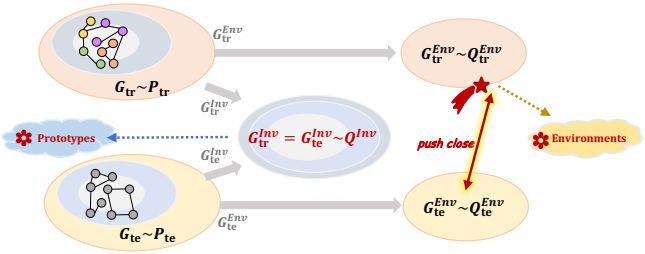


Fig. 2: Illustration of distribution shifts in test-time graphs v.s. training graphs.

where $\mathcal{L}^{\text{Inv}}(\cdot;\cdot)$ is influenced by the environment-invariant part, meanwhile, $\mathcal{L}^{\text{Env}}(\cdot;\cdot)$ captures the environment-varying part, with a discrepancy bounded by ϵ. Hence, the generalization error between G_{tr} and G_{te} can be expressed as:

$$\begin{aligned} &|\mathcal{L}_{\text{cls}}(G_{\text{tr}}; \boldsymbol{\theta}_{\text{tr}}) - \mathcal{L}_{\text{cls}}(G_{\text{te}}; \boldsymbol{\theta}_{\text{tr}})| \leq \\ &|\mathcal{L}^{\text{Inv}}(G_{\text{tr}}^{\text{Inv}}; \boldsymbol{\theta}_{\text{tr}}) - \mathcal{L}^{\text{Inv}}(G_{\text{te}}^{\text{Inv}}; \boldsymbol{\theta}_{\text{tr}})| + \\ &|\mathcal{L}^{\text{Env}}(G_{\text{tr}}^{\text{Env}}; \boldsymbol{\theta}_{\text{tr}}) - \mathcal{L}^{\text{Env}}(G_{\text{te}}^{\text{Env}}; \boldsymbol{\theta}_{\text{tr}})|. \end{aligned} \tag{4}$$

The first term generalizes well, since $G_{\text{tr}}^{\text{Inv}} = G_{\text{te}}^{\text{Inv}} \sim Q^{\text{Inv}}$, The second term is bounded by a small ϵ with $dist\left(G_{\text{tr}}^{\text{Env}}, G_{\text{te}}^{\text{Env}}\right) < \epsilon$, which dominates the total generalization error. This implies that reducing the discrepancy in environmental variations between the training and test graphs would enable $\text{GNN}_{\boldsymbol{\theta}^*_{\text{tr}}}$ to generalize effectively to G_{te}.

Theoretical Justification. We validate the rationale behind the assumption in Proposition. III.1, for addressing environment-varying variables with bounded discrepancy. The assumption that A = $dist\left(G_{\text{tr}}^{\text{Env}}, G_{\text{te}}^{\text{Env}}\right) < \epsilon$ is a **sufficient** but **non-necessary** condition for B =$\mathcal{L}_{\text{cls}}(G_{\text{te}}; \boldsymbol{\theta}^*_{\text{tr}}) < \omega$ with $A \to B$, where ω indicates $\text{GNN}_{\boldsymbol{\theta}^*_{\text{tr}}}$ model that has been well trained on the G_{tr} keeps good generalization with small generalization error ω.

- **Sufficiency**: Bounding the discrepancy $dist(G_{\text{tr}}^{\text{Env}}, G_{\text{te}}^{\text{Env}}) < \epsilon$ simplifies the problem and aligns with many practical scenarios, such as cases where environmental variations are minor perturbations or exhibit predictable patterns. In the ideal case, if there is no discrepancy between $(G_{\text{tr}}^{\text{Env}}, G_{\text{te}}^{\text{Env}})$, the distributional shift due to environment-varying components would be minimal, allowing the GNN model $\text{GNN}_{\boldsymbol{\theta}^*_{\text{tr}}}$ to naturally generalize well on $G_{\text{te}}^{\text{Env}}$.
- **Non-necessity**: The GNN model may still achieve good generalization even if this assumption is not fully satisfied. For example, if the environment-varying variables have limited influence on the model's predictions or if the model primarily relies on environment-invariant features G^{Inv}, the generalization error $\mathcal{L}_{\text{cls}}(G_{\text{te}}; \boldsymbol{\theta}^*_{\text{tr}}) < \omega$ may remain low even without bounding the discrepancy ϵ.

□

As shown in Fig. 2, we elaborate on the distribution shifts in the test-time graph and the training graph, from the view of latent variable decomposition according to Proposition 1. It

Authorized licensed use limited to: Hong Kong University of Science and Technology. Downloaded on September 07,2026 at 07:57:03 UTC from IEEE Xplore. Restrictions apply.

shows two fundamental insights and principles for test-time graph rebirth:

1) Re-extracting the environment-invariant features of the test-time graph, which shares the same informative characteristics with the training graph, to ensure that they can reflect the predictive pattern of node class labels, denoted as class-related **prototype** features.
2) Re-fining the environment-varying features, which are primarily attributed to possible shifts in the training-test distribution, referred to **environment** features. Essentially, a well-trained GNN model is supposed to perform expressively on the test graph, when the test distribution closely matches the training graph's distribution. When we push the test-time environment feature distributions more closely with those of the training graph, the well-trained GNN is likely to exhibit improved generalization capabilities under distribution shifts.

According to such two principles, if we (1) keep prototype features and (2) align the environment features on the test graph, then, make a re-composition, we could transform the original test graph to a new test graph. This process can be defined as the problem of test-time graph rebirth:

Definition III.2 (**Test-time Graph Rebirth**)**.** Given the test graph $G_{\text{te}} = (\mathbf{X}_{\text{te}}, \mathbf{A}_{\text{te}})$ and the well-trained GNN model $\text{GNN}_{\boldsymbol{\theta}^*_{\text{tr}}}$, test-time graph rebirth aims to learn following mapping functions: $f_{\text{Pro}} : G_{\text{te}} \to G^{\text{Inv}}_{\text{te}}$ and $f_{\text{Env}} : G_{\text{te}} \to G^{\text{Env}}_{\text{te}}$, with the re-composition function $g = f_{\text{Pro}} \circ f_{\text{Env}}$, the rebirth test graph can be denoted as

$$G'_{\text{te}} = (\mathbf{X}'_{\text{te}}, \mathbf{A}'_{\text{te}}) = g(f_{\text{Pro}}(G_{\text{te}}), f_{\text{Env}}(G_{\text{te}})). \tag{5}$$

In this way, the rebirth test graph would be fed into the well-trained GNN model that has been deployed online in practice for making the inference $\hat{\mathbf{Y}}'_{\text{te}} = \text{GNN}_{\boldsymbol{\theta}_{\text{tr}}}(\mathbf{X}'_{\text{te}}, \mathbf{A}'_{\text{te}})$, where $\hat{\mathbf{Y}}'_{\text{te}}$ is expected to be more closely aligned with the actual ground-truth node labels of the test graph compared to the initial predictions $\hat{\mathbf{Y}}_{\text{te}}$.

B. Methodology

The overall framework of our proposed TT-GREB is illustrated in Fig. 3. Concretely, the proposed TT-GREB consists of two components: (1) a prototype extractor identifies features that remain unchanged in different environments, mainly determined by node class labels, where these features can be consistent in both training and test graphs and reflect the predictive pattern of GNN models; and (2) an environment refiner adjusts the environment-varying features of the test-time graph, to match the latent distribution of the training graph's environment. This alignment ensures that the GNN, which is well-trained on the training graph, demonstrates strong generalization capability on the rebirth test graph. More details of the modular design of our proposed TT-GREB are presented below.

1) Modular Design: Given a test-time graph $G_{\text{te}} = (\mathbf{X}_{\text{te}}, \mathbf{A}_{\text{te}})$ with M nodes with d dimension node attribute features, the prototype extractor $f^{\text{Pro}}_{\boldsymbol{\phi}_p}(\cdot)$ and the environment refiner $f^{\text{Env}}_{\boldsymbol{\phi}_e}(\cdot)$, parameterized by $\boldsymbol{\phi_p}$ and $\boldsymbol{\phi_e}$, respectively, take it as the input simultaneously. For ease of reference, we denote them as $f_{\boldsymbol{\phi}_p}(\cdot)$ and $f_{\boldsymbol{\phi}_e}(\cdot)$, by omitting the superscripts in subsequent mentions.

Concretely, these two sub-modules share the same structure, *i.e.*, two full-connected layers, $\text{FC}^k_{\text{node}}(\cdot)$ and $\text{FC}^k_{\text{edge}}(\cdot)$ to generate the soft and dense node attribute reweight matrix $\mathbf{W}^k_{\text{node}} \in \mathbb{R}^{d\times d}$, and the edge reweight matrix $\mathbf{W}^k_{\text{edge}} \in \mathbb{R}^{M\times M}$, where $k = \{\text{Pro}, \text{Env}\}$ for indicating the layers in the prototype extractor and the environment refiner, respectively. In this way, we have:

$$\begin{aligned} \mathbf{W}^k_{\text{node}} &= \sigma\left(\text{FC}^k_{\text{node}}(\mathbf{X}_{\text{te}})\right), \\ \mathbf{W}^k_{\text{edge}}(i,j) &= \sigma\left(\text{FC}^k_{\text{edge}}\left(\left[\mathbf{x}^i_{\text{te}}, \mathbf{x}^j_{\text{te}}\right]\right)\right), \end{aligned} \tag{6}$$

where $\sigma(\cdot)$ is the sigmoid function that constrains both the node attribute reweight matrix and the edge reweight matrix to $[0, 1]$, and $\mathbf{x}^i_{\text{te}} = \mathbf{X}_{\text{te}}[i, :]$ denotes the i-th row's feature representation for node i, so as for node j with $\mathbf{x}^j_{\text{te}} = \mathbf{X}_{\text{te}}[j, :]$, attributing the value in the location (i, j) for $\mathbf{W}^k_{\text{edge}}$.

Then, we could obtain the graph $G_{\text{Pro}} = (\mathbf{X}_{\text{Pro}}, \mathbf{A}_{\text{Pro}})$ that reflects the node class label prototypes and the graph $G_{\text{Env}} = (\mathbf{X}_{\text{Env}}, \mathbf{A}_{\text{Env}})$ that adjusts the test-time graph characteristics that vary with environments under distribution shifts, as:

$$\begin{aligned} G_{\text{Pro}} &= f_{\boldsymbol{\phi}_p}(G_{\text{te}}) = \left(\mathbf{A}_{\text{te}} \odot \mathbf{W}^{\text{Pro}}_{\text{edge}},\ \mathbf{X}_{\text{te}} \odot \mathbf{W}^{\text{Pro}}_{\text{node}}\right), \\ G_{\text{Env}} &= f_{\boldsymbol{\phi}_e}(G_{\text{te}}) = \left(\mathbf{A}_{\text{te}} \odot \mathbf{W}^{\text{Env}}_{\text{edge}},\ \mathbf{X}_{\text{te}} \odot \mathbf{W}^{\text{Env}}_{\text{node}}\right), \end{aligned} \tag{7}$$

where $\odot$ is the broadcasted element-wise product. After these, we make a re-composition with the prototype graph G_{Pro} and the environment graph G_{Env} components to build a new test-time graph $G'_{\text{te}} = (\mathbf{X}'_{\text{te}}, \mathbf{A}'_{\text{te}})$ through:

$$\begin{aligned} &G'_{\text{te}} = g(G_{\text{te}}) = \left(\mathbf{A}_{\text{te}} \odot \mathbf{W}^{\text{Comp}}_{\text{edge}},\ \mathbf{X}_{\text{te}} \odot \mathbf{W}^{\text{Comp}}_{\text{node}}\right), \text{ where} \\ &\mathbf{W}^{\text{Comp}}_{\text{edge}} = \left(\mathbf{1}^{\text{edge}} - \mathbf{W}^{\text{Pro}}_{\text{edge}}\right) \odot \mathbf{W}^{\text{Env}}_{\text{edge}} + \mathbf{W}^{\text{Pro}}_{\text{edge}}, \text{ and} \\ &\mathbf{W}^{\text{Comp}}_{\text{node}} = \left(\mathbf{1}^{\text{node}} - \mathbf{W}^{\text{Pro}}_{\text{node}}\right) \odot \mathbf{W}^{\text{Env}}_{\text{node}} + \mathbf{W}^{\text{Pro}}_{\text{node}}. \end{aligned} \tag{8}$$

In this process, $\mathbf{1}^{\text{node}}$ and $\mathbf{1}^{\text{edge}}$ denote the all-one matrices. Given $\mathbf{W}^{\text{Pro}}_{\text{node}}$ represents the class-prototype node attribute reweight matrix, $\left(\mathbf{1}^{\text{node}} - \mathbf{W}^{\text{Pro}}_{\text{node}}\right)$ can be viewed as a plain and straightforward proportion of the environment-sensitive node attributes. Then, $\left(\mathbf{1}^{\text{node}} - \mathbf{W}^{\text{Pro}}_{\text{node}}\right) \odot \mathbf{W}^{\text{Env}}_{\text{node}}$ re-composes node attribute reweight matrix by explicitly imposing the environment refinement $\mathbf{W}^{\text{Env}}_{\text{node}}$ on the environment-sensitive proportion. And then, $+\mathbf{W}^{\text{Pro}}_{\text{node}}$ makes sure to preserve the environment consistent proportion. The edge reweight matrix composition would follow the same rule. The rationale for such a re-composition schema is based on the understanding that the interplay between node class label prototypes and environment features is typically more complex than a basic additive combination, such as $(\mathbf{W}^{\text{Env}}_{\text{node}} + \mathbf{W}^{\text{Pro}}_{\text{node}})$. This complexity becomes particularly evident under the distribution shifts encountered during test time.

By this re-composition schema, we jointly keep prototype features and align the environment features on the test graph, leading to a transformation from the original test graph to a

Authorized licensed use limited to: Hong Kong University of Science and Technology. Downloaded on September 07,2026 at 07:57:03 UTC from IEEE Xplore. Restrictions apply.

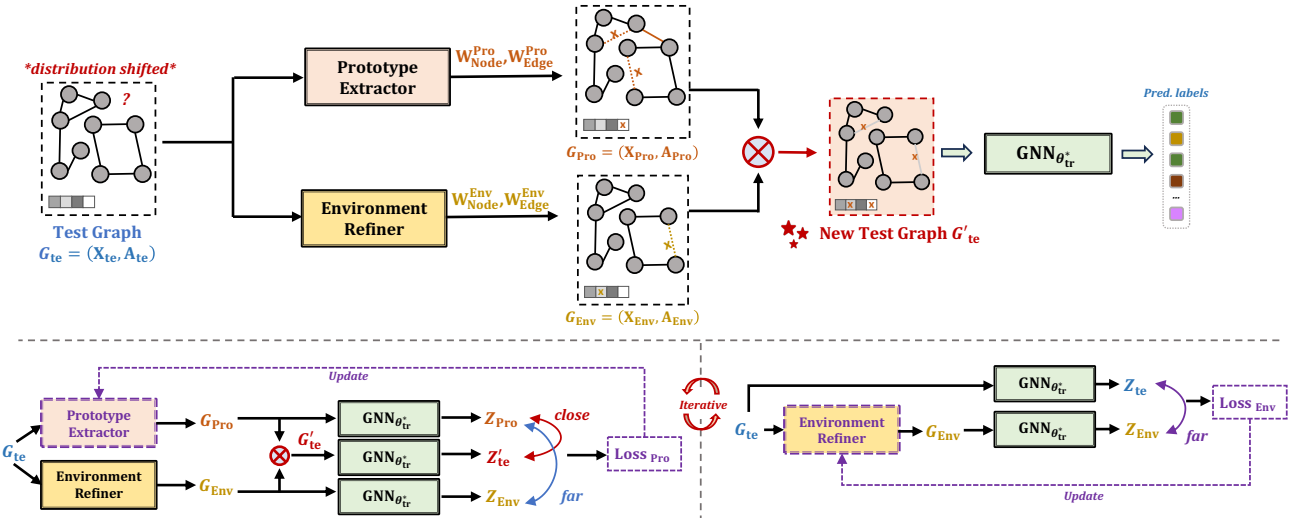


Fig. 3: The overall framework of the proposed test-time graph rebirth (TT-GREB) method.

newly reborn test graph. In this way, the new test-time rebirth graph can make effective predictions with good generalization ability on the well-trained GNN model with graph data distribution shifts.

2) Optimization Objective: During the test time, to improve the well-trained GNN model generalization under graph distribution shifts, the significant challenge faced with graph data-centric transformation through test-time graph rebirth is the scarcity of test ground-truth labels. Consequently, this makes it more challenging to conduct supervised learning by minimizing the cross-entropy loss, which is the most readily and straightforward solution. Therefore, with (1) the insufficient node class labels of the test graph, (2) the inaccessible training graph for online GNN deployment, and (3) the unknown graph distribution shifts, it is imperative to develop an effective self-supervised learning objective along with an appropriate optimization strategy.

In this work, we propose a dual test-time graph contrastive learning objective with an effective iterative optimization strategy. For one thing, we use self-supervised signals from the well-trained GNN's output node representations to guide the learning of the prototype extractor with the graph contrastive learning loss $\mathcal{L}_{\text{Pro}}$, following the parameter-free principle [26]. Through the lens of the general graph contrastive learning scheme, the core idea is to maximize the similarity between two consistent views of the same graph, and to minimize the similarity when the views are not in agreement. For another thing, we perform a decomposition of environment-varying features on the test graph, by ensuring the environmental discrepancy under the graph distribution shifts during the test time. Considering the inaccessible training graph, we encourage the discrepancy between the environmental characteristics of the reborn test-time graph and the original test-time graph, leading to the graph environment refinement loss $\mathcal{L}_{\text{Env}}$ to optimize the proposed environment refiner. As illustrated in the lower section of Fig.3, these two optimization objectives are iteratively refined using gradient descent until they reach convergence. More implementation details of the dual learning objective of our proposed TT-GREB are presented as follows.

Given the obtained prototype graph G_{Pro}, the environment graph G_{Env}, and the new rebirth graph G'_{te}, we fed them into the well-trained GNN model simultaneously, leading to the node representations with $\mathbf{Z}_{\text{Pro}} = \text{GNN}_{\boldsymbol{\theta}^*_{\text{tr}}}(G_{\text{Pro}})$, $\mathbf{Z}_{\text{Env}} = \text{GNN}_{\boldsymbol{\theta}^*_{\text{tr}}}(G_{\text{Env}})$, and $\mathbf{Z}_{\text{te}'} = \text{GNN}_{\boldsymbol{\theta}^*_{\text{tr}}}(G'_{\text{te}})$. Then, the learning objective of structural prototype feature extraction for test-time graph rebirth can be:

$$\begin{aligned}\min_{\boldsymbol{\phi}_p} \mathcal{L}_{\text{Pro}} = &\sum_{i=1}^{M}\left(1-\frac{(\mathbf{z}_i^{\text{te}'})^\top \mathbf{z}_i^{\text{Pro}}}{\left\|(\mathbf{z}_i^{\text{te}'})^\top\right\|\left\|\mathbf{z}_i^{\text{Pro}}\right\|}\right)\\ &-\sum_{i=1}^{N}\left(1-\frac{(\mathbf{z}_i^{\text{Env}})^\top \mathbf{z}_i^{\text{Pro}}}{\left\|(\mathbf{z}_i^{\text{Env}})\right\|\left\|\mathbf{z}_i^{\text{Pro}}\right\|}\right)\\ &+\alpha\left[Reg\left(\mathbf{W}^{\text{Pro}}_{\text{node}}\right)+Reg\left(\mathbf{W}^{\text{Pro}}_{\text{edge}}\right)\right].\end{aligned} \tag{9}$$

Furthermore, the learning objective of environment refinement can be written as:

$$\begin{aligned}\max_{\boldsymbol{\phi}_e} \mathcal{L}_{\text{Env}} = dist(G_{\text{te}}, G_{\text{Env}}) = ||\mathbf{Z}_{\text{te}} - \mathbf{Z}_{\text{Env}}||_2^2\\ -\beta\left[Reg\left(\mathbf{W}^{\text{Env}}_{\text{node}}\right)+Reg\left(\mathbf{W}^{\text{Env}}_{\text{edge}}\right)\right],\end{aligned} \tag{10}$$

where $Reg(\mathbf{W}^{\ddagger}) = |\frac{\sum \mathbf{W}^{\ddagger}}{\sum(1-\mathbf{W}^{\ddagger})} - \lambda_s|$ is the regularization term with $s = \{1,2,3,4\}$ and superscript‡ corresponding to $\mathbf{W}^{\text{Pro}}_{\text{node}}$, $\mathbf{W}^{\text{Pro}}_{\text{edge}}$, $\mathbf{W}^{\text{Env}}_{\text{node}}$, and $\mathbf{W}^{\text{Env}}_{\text{edge}}$, respectively. Here, λ_s acts as a hyper-parameter ranging $[0,1]$, while α and β balance the loss functions between the primary optimization objectives and the regularization terms. These regularization terms are designed to keep the average ratio of the number of reweighted node features or edges close to λ_s, thereby stabilizing the training process and avoiding trivial solutions.

IV. Experiment

In this section, we verify the effectiveness of the proposed TT-GREB in terms of the GNN generalization ability on test-time graphs under distribution shifts. Concretely, we aim to

Authorized licensed use limited to: Hong Kong University of Science and Technology. Downloaded on September 07,2026 at 07:57:03 UTC from IEEE Xplore. Restrictions apply.

TABLE I: Average classification results (%) over the test graphs under various graph distribution shifts on different backbone GNN models. The best results are in bold, and the second-best results are in underlines. 'Rank' indicates the average rank of each algorithm for each backbone; 'OOM' indicates an out-of-memory error on 32 GB GPU memory; TENT with '-' means it cannot be applied to GNNs without batch normalization layers.

Backbones	Categories	Methods	Amz-Photo	Cora	Elliptic	OGB-Arxiv	Twitch-E	Rank
GCN	Model-centric	ERM	88.60±0.90	87.49±7.97	51.09±5.63	38.39±2.92	59.80±3.77	3.8
		EERM	81.05±0.95	66.80±6.51	45.60±1.22	OOM	53.28±1.88	6
		DropEdge	81.73±1.23	74.05±8.00	53.83±4.52	**40.82±2.18**	59.49±4.14	3.8
		TENT	88.60±0.90	87.51±8.01	47.05±2.01	38.45±2.35	59.79±3.77	3.8
	Data-centric	GTRANS	**89.27±0.37**	95.20±0.87	56.69±6.74	40.00±2.30	60.38±3.86	1.8
		TT-GREB (Ours)	89.11±0.47	**96.12±1.10**	**57.20±8.19**	39.49±1.72	**60.85±4.17**	**1.6**
SAGE	Model-centric	ERM	84.03±7.61	98.48±3.68	57.34±5.95	39.26±2.39	62.08±4.04	4.4
		EERM	84.97±7.26	96.73±6.77	60.94±5.18	OOM	61.70±4.23	4.6
		DropEdge	80.67±1.61	92.53±7.12	52.84±3.92	37.90±1.74	62.19±4.16	4.8
		TENT	84.10±7.71	98.58±3.49	50.16±3.89	39.59±1.63	62.04±4.06	3.8
	Data-centric	GTRANS	**89.63±5.43**	**99.89±0.03**	62.54±7.94	39.49±2.34	62.04±4.06	2
		TT-GREB (Ours)	88.49±4.07	99.66±0.48	**66.97±8.94**	**40.15±1.65**	**62.43±4.26**	**1.4**
GAT	Model-centric	ERM	91.20±2.41	95.53±4.98	65.28±9.59	40.47±2.48	58.23±3.45	3.8
		EERM	89.13±4.06	87.04±11.07	50.40±3.48	OOM	**59.51±3.26**	4.6
		DropEdge	69.52±6.33	76.71±4.60	64.96±7.12	**43.91±1.93**	58.46±3.35	4
		TENT	91.40±2.36	95.57±4.96	56.86±5.10	30.36±1.20	58.23±3.45	4
	Data-centric	GTRANS	94.04±0.73	97.28±2.92	**66.85±9.80**	41.65±2.26	58.20±3.49	2.4
		TT-GREB (Ours)	**94.34±0.82**	**98.05±1.03**	66.05±8.92	41.45±2.00	58.53±3.50	**1.8**
GPR	Model-centric	ERM	87.04±2.86	87.24±9.11	64.79±7.26	44.38±2.97	59.77±3.73	3.4
		EERM	85.29±1.48	**89.50±7.83**	64.41±6.97	OOM	**61.76±4.06**	3
		DropEdge	74.20±6.90	73.29±10.19	60.62±6.06	43.96±2.37	59.89±3.99	4.6
		TENT	-	-	-	-	-	-
	Data-centric	GTRANS	86.94±2.62	87.45±8.91	67.65±10.49	**45.74±2.24**	59.89±3.61	2.4
		TT-GREB (Ours)	**88.55±1.68**	88.54±8.74	**71.34±10.01**	45.14±2.41	60.00±3.86	**1.6**

TABLE II: Dataset statistics with various test-time graph data distribution shifts.

Distribution shifts	Datasets	#Nodes	#Edges	#Classes	Metrics
Node feature shifts	Cora [47]	2,703	5,278	10	Accuracy
	Amazon-Photo [48]	7,650	119,081	10	Accuracy
Domain shifts	Twitch-E [49]	1,9129,498	31,299 - 153,138	2	ROC-AUC
Temporal shifts	Elliptic [50]	203,769	234,355	2	F1 Score
	OGB-arxiv [50]	169,343	1,166,243	40	Accuracy

answer the following questions to demonstrate the effectiveness of the proposed TT-GREB in terms of the GNN generalization ability on test-time graphs under distribution shifts: **Q1:** How does the proposed TT-GREB perform on the well-trained GNNs for the node classification task under various graph distribution shifts at test time? **Q2:** How does the proposed TT-GREB perform when conducting an ablation study regarding the sub-module components and the learning strategy? **Q3:** How sensitive are the hyper-parameter λ for the proposed TT-GREB? **Q4:** How does the proposed TT-GREB perform in terms of running time efficiency?

A. Experimental Settings

Datasets. We perform experiments on five real-world graph datasets with diverse graph data distribution shifts containing: node feature shifts: Cora [47] and Amazon-Photo [48]; domain shifts [29]: Twitch-E [49]; temporal shifts: Elliptic [50] and OGB-arxiv [50]. More statistical details of the datasets are listed in Table II. For all training, validation, and test graphs, we follow the process procedures and splits in previous works [19], [26], [29], [12]. For dataset splits of the number of training/validation/test graphs, we have Cora and Amazon-Photo for 1/1/8, Twitch-E for 1/1/5, Elliptic for 5/5/33, and OGB-arxiv for 1/1/3.

TABLE III: Performance comparison of MATCHA (*graph homophily structural shift specific method*) vs. our proposed TT-GREB (*universal graph distribution shift method*) across different backbones and datasets in terms of average classification results (%). Bold indicates the best result.

Methods	Amz-Photo	Cora	Elliptic	OGB-Arxiv	Twitch-E
			GCN		
MATCHA	88.56±0.86	89.57±2.54	54.80±3.16	**40.30**±3.54	59.98±7.61
TT-GREB (Ours)	**89.11**±0.37	**96.12**±1.10	**57.20**±8.19	39.49±1.72	**60.85**±4.17
			SAGE		
MATCHA	86.62±2.45	95.96±2.27	59.39±2.85	39.66±1.71	62.22±6.09
TT-GREB (Ours)	**88.49**±4.07	**99.66**±0.48	**66.97**±8.94	**40.15**±1.65	**62.43**±4.26
			GAT		
MATCHA	92.87±3.34	91.03±2.88	65.48±3.36	**44.97**±2.43	**59.20**±2.94
TT-GREB (Ours)	**94.34**±0.82	**98.05**±1.93	**66.05**±8.92	41.45±2.00	58.53±3.50
			GPR		
MATCHA	88.11±2.90	88.41±9.43	65.70±3.22	**45.33**±1.22	**60.30**±6.84
TT-GREB (Ours)	**88.55**±1.68	**88.54**±8.47	**71.34**±10.01	45.14±2.41	60.00±3.86

Test-time Evaluation Protocol. We test four commonly used GNN models for evaluating GNN generalization under graph distribution shifts following the settings in [26], including GCN [51], GraphSAGE [52] (*abbr.* SAGE), GAT [53], and GPR-GNN [54] (*abbr.* GPR). For each model, we train it

Authorized licensed use limited to: Hong Kong University of Science and Technology. Downloaded on September 07,2026 at 07:57:03 UTC from IEEE Xplore. Restrictions apply.

on training sets, until the model achieves the optimal node classification on its validation sets, following the standard training process, so that we can obtain the 'well-trained' GNN model that remains fixed in the whole test-time graph rebirth process. We report the average classification performance, and for all experiments, we report the average results of 10 repeated times with different random seeds.

Baseline Methods. We compare the proposed TT-GREB with the following baselines that fall in two groups: *graph model-centric methods*: empirical risk minimization (ERM) for standard training [19], data augmentation technique DropEdge [55], Explore-to-Extrapolate Risk Minimization (EERM) [19] customized for node-level graph OOD generalization, and test-time training method TENT [35]; And the recent SOTA *graph data-centric method*: test-time graph transformation method GTRANS [26]. Besides, we also conduct the performance comparison between the very recent test-time adaptation method MATCHA [39] and our proposed TT-GREB.

B. Experimental Results

In Table I, we report the average node-level classification results over the test graphs under various graph distribution shifts on different backbone GNN models, along with the average rank of each comparison method for each backbone.

As can be observed, our proposed TT-GREB generally delivers great performance across various graph datasets and models, achieving the highest ranks overall: 1.6, 1.4, 1.8, and 1.6 for GCN, SAGE, GAT, and GPR, respectively. These results could verify the outstanding effectiveness of the proposed TT-GREB for modifying graph data at test time to serve better GNN generalization ability. Moreover, compared with the recent SOTA graph data-centric method GTRANS, our proposed TT-GREB achieves significant improvements in some cases: for example, our method has 5.5% average improvement of the classification performance from GTRANS's 67.65% to 71.34% on Elliptic dataset with GPR model, and 7.1% average improvement with SAGE model, respectively.

Besides, we can also observe that some model-centric comparison methods, achieve great performance in some cases, for instance, DropEdge and EERM could deliver excellent performance on OGB-arxiv with GCN and Twitch-e with GAT models, respectively. Nevertheless, DropEdge and EERM can not be applied to the test-time application scenario, since it has to modify the training process of GNNs to achieve better generalization ability. Although TENT is suited for the test-time adaptation and training scenario, it can not be directly used for models without batch normalization layers, significantly limiting its usage on GNN models.

In Table III, we present the average classification performance of MATCHA [39], a method tailored specifically for structural shifts in graph homophily, compared to our proposed TT-GREB, a universal method designed to handle general graph distribution shifts. The results demonstrate the overall superior performance of TT-GREB, which outperforms MATCHA in 15 out of 20 test cases. The advantage of MATCHA for the remaining 5 cases stems from being specifically optimized for homophilous graphs and certain GNN backbones (*e.g.*, GPR). Besides, we also include an architecture backbone of recent graph transformer [56] to further strengthen our evaluation for test-time graph modification in Table IV.

TABLE IV: Test-time performance comparison on the Graph Transformer (SGFformer [56]) backbone.

Methods	Amz-Photo	Cora	Elliptic	OGB-Arxiv	Twitch-E
ERM	89.10±4.49	81.29±17.83	68.79±8.36	42.92±2.61	61.85±4.02
TENT	74.50±3.87	86.71±15.23	44.06±13.01	43.11±0.50	60.36±3.55
GTRANS	85.06±1.27	84.24±3.71	17.15±2.13	32.76±1.01	58.80±2.99
TT-GREB (Ours)	**91.27**±3.13	**84.01**±17.40	**72.82**±8.95	**43.45**±2.41	**62.05**±3.97

TABLE V: Ablation study components of the proposed method. For Idx02, ✓* denotes enabling the complete framework in the test-time graph rebirth process but only using the output G_{Pro} of the prototype extractor for final inference.

Ablation Index	Sub-module Components		Learning Strategies	
	ProExtractor	EnvRefiner	Contrastive_Obj	Iterative_Opt
Baseline	×	×	×	×
Idx00	✓	×	×	×
Idx01	✓	✓	✓	×
Idx02	✓*	✓	✓	✓
Idx03 (TT-GREB)	✓	✓	✓	✓

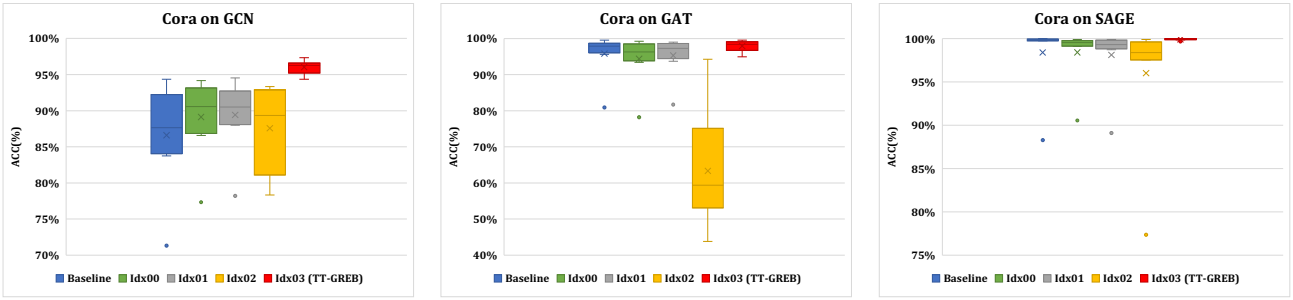
In summary, our proposed TT-GREB significantly improves GNN generalization for different graph distribution shifts and GNN models during test time, achieving superior average rankings compared to existing approaches. This success is due to the collaboration between the prototype extractor and environment refiner, which enhances the representations of test graph nodes and edges. Additionally, the incorporation of a dual graph contrastive learning objective, coupled with an effective iterative optimization strategy, further contributes to the method's outstanding performance.

C. Ablation Study of TT-GREB

In Table V, we evaluate the effectiveness of the overall framework of the proposed TT-GREB, from the perspectives of sub-module components and learning strategies, respectively. We observe the effectiveness of the prototype extractor (ProExtractor), and the environment refiner (EnvRefiner), respectively. For learning strategies, we test the effectiveness of with and without the dual graph contrastive learning objective (Contrastive_Obj) as well as the iterative optimization method (Iterative_Opt). Baseline denotes the original test graph classification performance directly inferring on the well-trained GNNs without any test-time modification. For Idx00 without the contrastive learning objective, the optimization objective would be degraded to the close distance constraint between the output prototype extractor and the original test graph, which means with the weakest supervision signals to instruct the learning process. For Idx02 with ✓*, it denotes that we use the overall proposed framework to give a test-time graph rebirth, but only access the partial output, *i.e.*, the output G_{Pro} of the prototype extractor for final test-time inference.

The results on all test graphs of Cora on GCN and GAT with a fixed seed run are presented in Fig. 4 (a) and Fig. 4

Authorized licensed use limited to: Hong Kong University of Science and Technology. Downloaded on September 07,2026 at 07:57:03 UTC from IEEE Xplore. Restrictions apply.

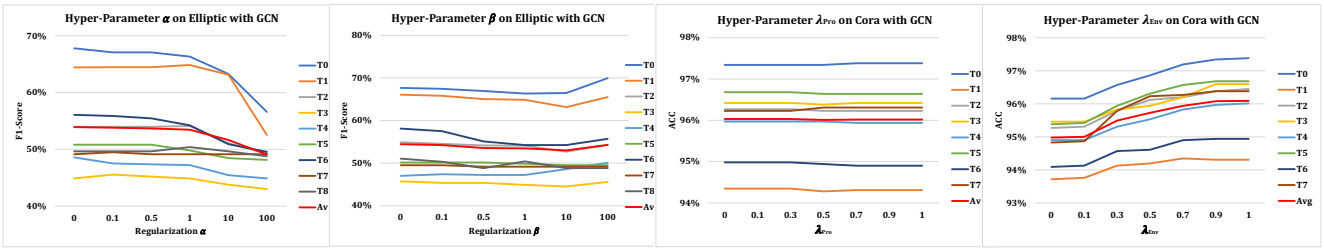


(a) Ablation study results (%) on Cora with GCN in Box-plot.

(b) Ablation study results (%) on Cora with GAT in Box-plot.

(c) Ablation study results (%) on Cora with SAGE in Box-plot.

Fig. 4: In-depth analysis of our proposed TT-GREB in terms of ablation study results.



(a) α on Elliptic (F1-Score%).

(b) β on Elliptic (F1-Score%).

(c) λ_{Pro} on Cora (ACC%).

(d) λ_{Env} on Cora (ACC%).

Fig. 5: Hyper-parameter sensitivity analysis of GCN models on Elliptic and Cora datasets.

(b). As can be observed, our proposed TT-GREB achieves consistently good classification performance on all test graphs on average, also with the smallest standard deviations across all models. Besides, it can also be observed that, generally, each component of sub-modules and learning strategies could contribute to performance improvement to different degrees when these components are coupled together to achieve the best performance of the proposed TT-GREB.

D. Hyper-parameter Sensitivity Analysis

In Fig. 5, we evaluate the sensitivity of all hyper-parameters in terms of the regularization in our proposed TT-GREB in Eq. (9) and Eq. (10). Concretely, λ_{Pro} indicates $s = \{1, 2\}$ in Eq. (9) and Eq. (10) for $\lambda_1 = \lambda_2$, corresponding to $\mathbf{W}^{\text{Pro}}_{\text{node}}$, $\mathbf{W}^{\text{Pro}}_{\text{edge}}$, and we empirically set the $\lambda_3 = 1$ for $\mathbf{W}^{\text{Env}}_{\text{node}}$. And λ_{Env} indicates λ_4, corresponding to $\mathbf{W}^{\text{Pro}}_{\text{edge}}$. For regularization weights in balancing the optimization objective, we observe α and β in a set of $\{0, 0.1, 0.5, 1, 10, 100\}$. For λ_{Pro} and λ_{Env}, we observe the parameter range of $[0, 1]$ with an interval of 0.1. Observations indicate that the hyper-parameters demonstrate a moderate level of sensitivity within specific ranges, underscoring the robustness of our proposed method to hyper-parameter tuning.

E. Computation Complexity & Running Time

Given the data-centric nature of test-time graph transformations, the computational complexity scales with the size of the graph and is independent of the design of our model. We provide a detailed analysis in terms of computational intensity of our proposed iterative optimization strategy as follows:

TABLE VI: Running time (seconds) comparison in 20 epochs with a single NVIDIA A100 GPU on all graph distribution shift datasets with GCN model.

Methods	Amz-Photo	Cora	Elliptic	OGB-Arxiv	Twitch-E
EERM	14.66	2.67	230.50	191.48	22.14
GTRANS	0.24	0.13	0.32	0.89	0.20
TT-GREB (Ours)	2.06	1.08	1.53	4.29	1.76

- **Reweighting Edges**: Operations over $\boldsymbol{W}_{\text{edge}} \in \mathbb{R}^{N \times N}$ scale with $\mathcal{O}(N^2)$. For graphs with a very large number of nodes N, this can still become computationally intensive.
- **Iterative Learning**: The iterative updates add a scaling factor T (number of iterations) to the complexity, where each iteration involves: (a) Forward and backward passes through the GNN, scaling with $\mathcal{O}(NFd + NF^2 + N^2F)$; and (b) Gradient updates and regularization for reweighting matrices $\mathcal{O}(N^2 + d^2)$.

These computations ensure that our proposed TT-GREB is practical for moderately large graphs. However, the computational requirements for larger graphs are influenced by the data characteristics inherent to the node-level graph learning task.

In Table VI, we compare the running time of our proposed TT-GREB with existing baseline methods, *i.e.*, EERM and GTRANS, which are specifically designed for the graph distribution shift issue. The results are obtained in a single NVIDIA A100 GPU across all datasets with the GCN model in 20 epochs. It can be observed that our proposed method achieves a comparable running time with GTRANS, and significantly exceeds the EERM method, demonstrating its great time efficiency. This efficiency stems primarily from

Authorized licensed use limited to: Hong Kong University of Science and Technology. Downloaded on September 07,2026 at 07:57:03 UTC from IEEE Xplore. Restrictions apply.

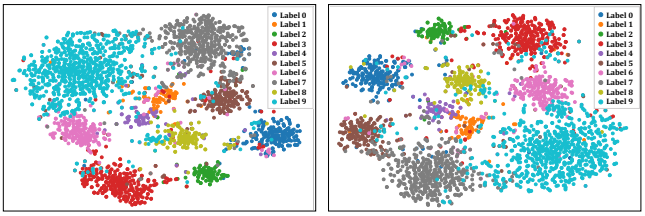


Fig. 6: Visualization comparison of t-SNE on the embeddings of the original test graph (1-st test graph) and our reborn test graph with the well-trained GCN model on Cora.

the fact that EERM, a graph data-centric method, necessitates retraining the GNN, which is inherently time-consuming. In contrast, both GTRANS and our proposed method employ test-time graph modifications to enhance performance. However, our method incurs a slight increase in time consumption due to the implementation of a dual iterative optimization strategy.

F. Visualization Results

For a comprehensive understanding of our reborn test graph, in Fig. 6, we present the t-SNE visualization of the original test graph and our reborn test graph on Cora's first test graph, in terms of the output node representations of the well-trained GCN model. It can be seen that the clusters corresponding to different node class labels are more distinctly separated in the t-SNE latent space after experiencing our proposed test-graph rebirth process. This could effectively verify that the proposed TT-GREB can be beneficial to improve node representation learning, contributing better separation in the latent space of the test data, demonstrating good generalization ability of our method under graph distribution shifts.

V. Conclusion

In this work, we proposed a new graph data-centric method, test-time graph rebirth (TT-GREB), aimed at enhancing the generalization ability of GNN models to test-time graphs affected by distribution shifts through direct manipulation of the test graph data. The overall framework includes a prototype extractor for learning environment-invariant features and an environment refiner for adjusting environment-sensitive features, followed by a dual test-time graph contrastive learning objective and an efficient iterative optimization strategy, facilitating the extraction of optimal prototype and environmental components of the reborn test graph. Our extensive experiments on real-world graph datasets under various test-time distribution shifts confirm the superiority of our method, underscoring its innovative capacity to modify test-time graphs for enhanced GNN generalization. A potential limitation of this work is its current focus on node-level tasks, but future extensions are expected to adapt it for broader applications in graph-level and edge-level tasks.

Acknowledgments

Shirui Pan was supported in part by the Australian Research Council (ARC) under grants FT210100097 and DP240101547 and the CSIRO – National Science Foundation (US) AI Research Collaboration Program (IIS-2302786). Qin Zhang was supported by National Natural Science Foundation of China (62576221, 62206179) and Guangdong Provincial Natural Science Foundation (2025A1515010288).

References

[1] S. Pan, L. Luo, Y. Wang, C. Chen, J. Wang, and X. Wu, "Unifying large language models and knowledge graphs: A roadmap," *IEEE Transactions on Knowledge and Data Engineering*, vol. 36, no. 7, pp. 3580–3599, 2024.

[2] H. Y. Koh, Y. Zheng, M. Yang, R. Arora, G. I. Webb, S. Pan, L. Li, and G. M. Church, "Ai-driven protein design," *Nature Reviews Bioengineering*, pp. 1–23, 2025.

[3] H. Zhang, B. Wu, X. Yuan, S. Pan, H. Tong, and J. Pei, "Trustworthy graph neural networks: Aspects, methods, and trends," *Proc. IEEE*, vol. 112, no. 2, pp. 97–139, 2024.

[4] X. Zheng, Y. Liu, S. Pan, M. Zhang, D. Jin, and P. S. Yu, "Graph neural networks for graphs with heterophily: A survey," *arXiv preprint arXiv:2202.07082*, 2022.

[5] X. Zheng, M. Zhang, C. Chen, C. Li, C. Zhou, and S. Pan, "Multi-relational graph neural architecture search with fine-grained message passing," in *2022 IEEE International Conference on Data Mining (ICDM)*. IEEE, 2022, pp. 783–792.

[6] X. Zheng, M. Zhang, C. Chen, Q. V. H. Nguyen, X. Zhu, and S. Pan, "Structure-free graph condensation: From large-scale graphs to condensed graph-free data," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

[7] X. Zheng, Y. Liu, Z. Bao, M. Fang, X. Hu, A. W.-C. Liew, and S. Pan, "Towards data-centric graph machine learning: Review and outlook," *arXiv preprint arXiv:2309.10979*, 2023.

[8] Y. Liu, G. Zhang, K. Wang, S. Li, and S. Pan, "Graph-augmented large language model agents: Current progress and future prospects," *arXiv preprint arXiv:2507.21407*, 2025.

[9] S. Li, Y. Liu, Q. Wen, C. Zhang, and S. Pan, "Assemble your crew: Automatic multi-agent communication topology design via autoregressive graph generation," *arXiv preprint arXiv:2507.18224*, 2025.

[10] H. Zhang, B. Wu, X. Yang, X. Yuan, X. Liu, and X. Yi, "Dynamic graph unlearning: A general and efficient post-processing method via gradient transformation," in *Proceedings of The Web Conference (WWW)*. ACM, 2025, pp. 931–944.

[11] F. Ke, J. Hsu, Z. Cai, Z. Ma, X. Zheng, X. Wu, S. Huang, W. Wang, P. D. Haghighi, G. Haffari *et al.*, "Explain before you answer: A survey on compositional visual reasoning," *arXiv preprint arXiv:2508.17298*, 2025.

[12] X. Zheng, M. Zhang, C. Chen, S. Molaei, C. Zhou, and S. Pan, "Gnnevaluator: Evaluating gnn performance on unseen graphs without labels," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

[13] Y. Huang, Z. Zhang, Q. Zhao, X. Yuan, and C. Chen, "Themis: Towards practical intellectual property protection for post-deployment on-device deep learning models," in *34th USENIX Security Symposium (USENIX Security 25)*, 2025.

[14] J. Pan, Y. Liu, X. Zheng, Y. Zheng, A. W.-C. Liew, F. Li, and S. Pan, "A label-free heterophily-guided approach for unsupervised graph fraud detection," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 39, no. 12, 2025, pp. 12 443–12 451.

[15] Y. Zheng, H. Zhang, V. Lee, Y. Zheng, X. Wang, and S. Pan, "Finding the missing-half: Graph complementary learning for homophily-prone and heterophily-prone graphs," in *International Conference on Machine Learning*. PMLR, 2023, pp. 42 492–42 505.

[16] Y. Zheng, S. Pan, V. C. Lee, Y. Zheng, and P. S. Yu, "Rethinking and scaling up graph contrastive learning: An extremely efficient approach with group discrimination," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.

[17] X. Shen, Y. Liu, Y. Dai, Y. Wang, R. Miao, Y. Tan, S. Pan, and X. Wang, "Understanding the information propagation effects of communication topologies in llm-based multi-agent systems," in *EMNLP*, 2025.

[18] X. Zheng, D. Song, Q. Wen, B. Du, and S. Pan, "Online gnn evaluation under test-time graph distribution shifts," in *The Twelfth International Conference on Learning Representations (ICLR)*, 2024.

Authorized licensed use limited to: Hong Kong University of Science and Technology. Downloaded on September 07,2026 at 07:57:03 UTC from IEEE Xplore. Restrictions apply.

[19] Q. Wu, H. Zhang, J. Yan, and D. Wipf, "Handling distribution shifts on graphs: An invariance perspective," in *International Conference on Learning Representations (ICLR)*, 2022.

[20] Y. Chen, Y. Bian, K. Zhou, B. Xie, B. Han, and J. Cheng, "Does invariant graph learning via environment augmentation learn invariance?" in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

[21] M. Wu, X. Zheng, Q. Zhang, X. Shen, X. Luo, X. Zhu, and S. Pan, "Graph learning under distribution shifts: A comprehensive survey on domain adaptation, out-of-distribution, and continual learning," *arXiv preprint arXiv:2402.16374*, 2024.

[22] J. Yu, J. Liang, and R. He, "Mind the label shift of augmentation-based graph ood generalization," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2023, pp. 11 620–11 630.

[23] X. Zheng, W. Huang, C. Zhou, M. Li, and S. Pan, "Test-time graph neural dataset search with generative projection," in *Forty-second International Conference on Machine Learning (ICML)*, 2025.

[24] Y. Liu, X. Zheng, Y. Li, and Y. Guo, "Test-time adaptation on recommender system with data-centric graph transformation," *International Joint Conference on Artificial Intelligence (IJCAI)*, 2025.

[25] Y. Sui, Q. Wu, J. Wu, Q. Cui, L. Li, J. Zhou, X. Wang, and X. He, "Unleashing the power of graph data augmentation on covariate distribution shift," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

[26] W. Jin, T. Zhao, J. Ding, Y. Liu, J. Tang, and N. Shah, "Empowering graph representation learning with test-time graph transformation," in *International Conference on Learning Representations (ICLR)*, 2023.

[27] Y. You, T. Chen, Z. Wang, and Y. Shen, "Graph domain adaptation via theory-grounded spectral regularization," in *International Conference on Learning Representations (ICLR)*, 2023.

[28] Y. Chen, W. Huang, K. Zhou, Y. Bian, B. Han, and J. Cheng, "Towards understanding feature learning in out-of-distribution generalization," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

[29] M. Wu, S. Pan, C. Zhou, X. Chang, and X. Zhu, "Unsupervised domain adaptive graph convolutional networks," in *Proceedings of The Web Conference (WWW)*, 2020, pp. 1457–1467.

[30] K. Xu, H. Chen, S. Liu, P. Chen, T. Weng, M. Hong, and X. Lin, "Topology attack and defense for graph neural networks: An optimization perspective," in *Proceedings of International Joint Conference on Artificial Intelligence (IJCAI)*, 2019.

[31] Q. Zhu, N. Ponomareva, J. Han, and B. Perozzi, "Shift-robust gnns: Overcoming the limitations of localized graph training data," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 34, 2021, pp. 27 965–27 977.

[32] H. Liu, B. Hu, X. Wang, C. Shi, Z. Zhang, and J. Zhou, "Confidence may cheat: Self-training on graph neural networks under distribution shift," in *Proceedings of the ACM Web Conference 2022*, 2022, pp. 1248–1258.

[33] M. Wu, S. Pan, and X. Zhu, "Attraction and repulsion: Unsupervised domain adaptive graph contrastive learning network," *IEEE Transactions on Emerging Topics in Computational Intelligence*, vol. 6, no. 5, pp. 1079–1091, 2022.

[34] Y. Sun, X. Wang, Z. Liu, J. Miller, A. Efros, and M. Hardt, "Test-time training with self-supervision for generalization under distribution shifts," in *International Conference on Machine Learning*. PMLR, 2020, pp. 9229–9248.

[35] D. Wang, E. Shelhamer, S. Liu, B. Olshausen, and T. Darrell, "Tent: Fully test-time adaptation by entropy minimization," in *International Conference on Learning Representations (ICLR)*, 2020.

[36] L. Chen, Y. Zhang, Y. Song, Y. Shan, and L. Liu, "Improved test-time adaptation for domain generalization," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2023, pp. 24 172–24 182.

[37] J. Liang, D. Hu, and J. Feng, "Do we really need to access the source data? source hypothesis transfer for unsupervised domain adaptation," in *International Conference on Machine Learning (ICML)*. PMLR, 2020, pp. 6028–6039.

[38] M. Jang, S.-Y. Chung, and H. W. Chung, "Test-time adaptation via self-training with nearest neighbor information," in *International Conference on Learning Representations (ICLR)*, 2022.

[39] W. Bao, Z. Zeng, Z. Liu, H. Tong, and J. He, "Matcha: Mitigating graph structure shifts with test-time adaptation," in *The Thirteenth International Conference on Learning Representations (ICLR)*, 2025.

[40] Y. Guo, C. Yang, Y. Chen, J. Liu, C. Shi, and J. Du, "A data-centric framework to endow graph neural networks with out-of-distribution detection ability," in *Proceedings of ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD)*, A. K. Singh, Y. Sun, L. Akoglu, D. Gunopulos, X. Yan, R. Kumar, F. Ozcan, and J. Ye, Eds. ACM, 2023, pp. 638–648.

[41] G. Chen, J. Zhang, X. Xiao, and Y. Li, "Graphtta: Test time adaptation on graph neural networks," *arXiv preprint arXiv:2208.09126*, 2022.

[42] Y. Wang, C. Li, W. Jin, R. Li, J. Zhao, J. Tang, and X. Xie, "Test-time training for graph neural networks," *arXiv preprint arXiv:2210.08813*, 2022.

[43] S. Gui, X. Li, L. Wang, and S. Ji, "Good: A graph out-of-distribution benchmark," *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 35, pp. 2059–2073, 2022.

[44] N. Ye, K. Li, H. Bai, R. Yu, L. Hong, F. Zhou, Z. Li, and J. Zhu, "Ood-bench: Quantifying and understanding two dimensions of out-of-distribution generalization," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2022, pp. 7947–7958.

[45] Y. Wu, X. Wang, A. Zhang, X. He, and T.-S. Chua, "Discovering invariant rationales for graph neural networks," in *International Conference on Learning Representations (ICLR)*, 2021.

[46] Y. Chen, Y. Zhang, Y. Bian, H. Yang, M. Kaili, B. Xie, T. Liu, B. Han, and J. Cheng, "Learning causally invariant representations for out-of-distribution generalization on graphs," *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 35, pp. 22 131–22 148, 2022.

[47] Z. Yang, W. Cohen, and R. Salakhudinov, "Revisiting semi-supervised learning with graph embeddings," in *International Conference on Machine Learning (ICML)*. PMLR, 2016, pp. 40–48.

[48] O. Shchur, M. Mumme, A. Bojchevski, and S. Günnemann, "Pitfalls of graph neural network evaluation," *arXiv preprint arXiv:1811.05868*, 2018.

[49] B. Rozemberczki, C. Allen, and R. Sarkar, "Multi-scale attributed node embedding," *Journal of Complex Networks*, vol. 9, no. 2, p. cnab014, 2021.

[50] A. Pareja, G. Domeniconi, J. Chen, T. Ma, T. Suzumura, H. Kanezashi, T. Kaler, T. Schardl, and C. Leiserson, "Evolvegcn: Evolving graph convolutional networks for dynamic graphs," in *Proceedings of AAAI Conference on Artificial Intelligence*, vol. 34, no. 04, 2020, pp. 5363–5370.

[51] T. N. Kipf and M. Welling, "Semi-supervised classification with graph convolutional networks," in *International Conference on Learning Representations (ICLR)*, 2017.

[52] W. Hamilton, Z. Ying, and J. Leskovec, "Inductive representation learning on large graphs," *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.

[53] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Liò, and Y. Bengio, "Graph attention networks," in *International Conference on Learning Representations (ICLR)*, 2017.

[54] E. Chien, J. Peng, P. Li, and O. Milenkovic, "Adaptive universal generalized pagerank graph neural network," in *International Conference on Learning Representations (ICLR)*, 2020.

[55] Y. Rong, W. Huang, T. Xu, and J. Huang, "Dropedge: Towards deep graph convolutional networks on node classification," in *International Conference on Learning Representations (ICLR)*, 2019.

[56] Q. Wu, W. Zhao, C. Yang, H. Zhang, F. Nie, H. Jiang, Y. Bian, and J. Yan, "Sgformer: Simplifying and empowering transformers for large-graph representations," *Advances in Neural Information Processing Systems*, vol. 36, pp. 64 753–64 773, 2023.

Authorized licensed use limited to: Hong Kong University of Science and Technology. Downloaded on September 07,2026 at 07:57:03 UTC from IEEE Xplore. Restrictions apply.